Bangladesh University of Business and Technology

Department of Computer Science and Engineering

PROJECT REPORT

Online Examination Portal

A Deliberately Vulnerable Web Application

Course Code	CSE 414
Course Title	Cyber Security and Digital Forensic Lab
Date of Submission	16 July 2026

SUBMITTED TO

Humayan Kabir Rupak

Lecturer

Department of Computer Science and Engineering

SUBMITTED BY

Sultan Ahmmed

ID: 22235103227

Intake 51, Section 05

Department of Computer Science and Engineering

Table of Contents

1. Introduction

The Online Examination Portal is a web application built for this course with a set of common security flaws left in on purpose. It simulates an academic platform: students can log in, take exams, view grades, and post in a discussion forum, while an administrator manages users, publishes exams, and reviews analytics.

The goal of the project is to show how well-known web vulnerabilities show up inside ordinary application features, how an attacker can exploit each one, and why each works at a technical level. Four vulnerability classes are covered: SQL Injection (SQLi), Stored Cross-Site Scripting (Stored XSS), Reflected Cross-Site Scripting (Reflected XSS), and Cross-Site Request Forgery (CSRF).

Each vulnerability sits inside a real feature of the portal instead of being bolted on separately, so the exploitation looks like what happens in a live application. The portal itself is built in PHP with a MySQL/MariaDB database, and it runs on Kali Linux behind Apache.

2. Application Features

The portal supports two authenticated user roles and one public, unauthenticated mode.

User roles

- Student: a registered learner who can log in, view available exams, take assessments, check grades, update their profile, and post in the discussion forum.
- Admin: an administrator who can view all students, delete student accounts, create and delete exams, and view portal analytics.
- Public (no login): anyone can view the homepage, search for student profiles, and view public profile pages without signing in.

Core functionality (CRUD)

The application performs full Create, Read, Update, and Delete operations against the database in real time: admins create and delete exams and students, students read exams and create forum comments, and the public can read student profiles.

Database structure

The application uses a MySQL database named sqli with four tables:

- users: stores all accounts (admins and students), including username, password, email, and role. Access control is decided by the role column.
- exams: stores every exam, including a title, description, the creator's ID, and a timestamp.
- comments: stores discussion forum posts, each linked to a specific user through the user_id foreign key.
- grades: stores student marks for various subjects, linked to the student through the student_id foreign key.

3. Methodology

Development: a functional exam portal was built in PHP with a MySQL backend, with the target vulnerabilities placed deliberately inside normal application features: login, profile viewing, discussion posting, search, and admin management.

Deployment: the application was deployed on Kali Linux. The project files were placed under the Apache web root, the database schema and seed data (setup.sql) were imported into MySQL, and the Apache and MySQL services were started.

Manual exploitation: each vulnerability was first exploited by hand through the browser, to confirm it was present and to see how it behaved.

Automated exploitation: for the URL-based SQL injection, sqlmap was used to confirm the flaw and pull data out of the database, showing how the same flaw could be exploited at scale.

Analysis: for each vulnerability, the underlying cause in the source code was identified and explained.

4. Tools and Environment

- Kali Linux: the operating system used to host and attack the application.
- Apache HTTP Server: served the PHP application.
- MySQL / MariaDB: the relational database storing all application data.
- PHP: the server-side language the application is written in.
- Firefox / Chrome: the web browser used for manual testing and exploitation.
- sqlmap: an automated SQL injection and database takeover tool, used against the URL-based injection point.

5. Attacks and Exploitation

This section is the core of the report. Each vulnerability is presented with its location in the application, the payload used, the steps to reproduce it, an explanation of why it works, and placeholders for the corresponding screenshots.

5.1 SQL Injection: Login Form (POST)

Location: login.php, the username and password login form.

Type: in-band SQL injection used for authentication bypass.

Payload:

```
Username: admin' –
Password: anything
```

Steps:

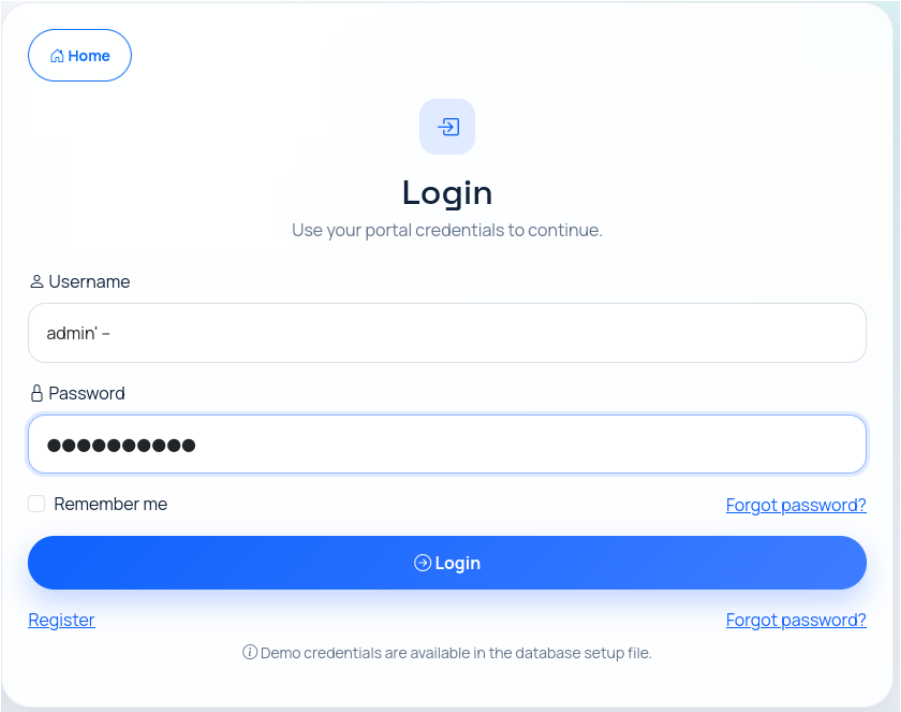
- Open the login page at http://localhost/login.php.
- In the username field enter: admin' -- (note the trailing space).
- Enter any value in the password field and submit.
- The application logs in as the admin account without a valid password and redirects to the admin dashboard.

Why it works: the login query is built by directly concatenating user input into the SQL string:

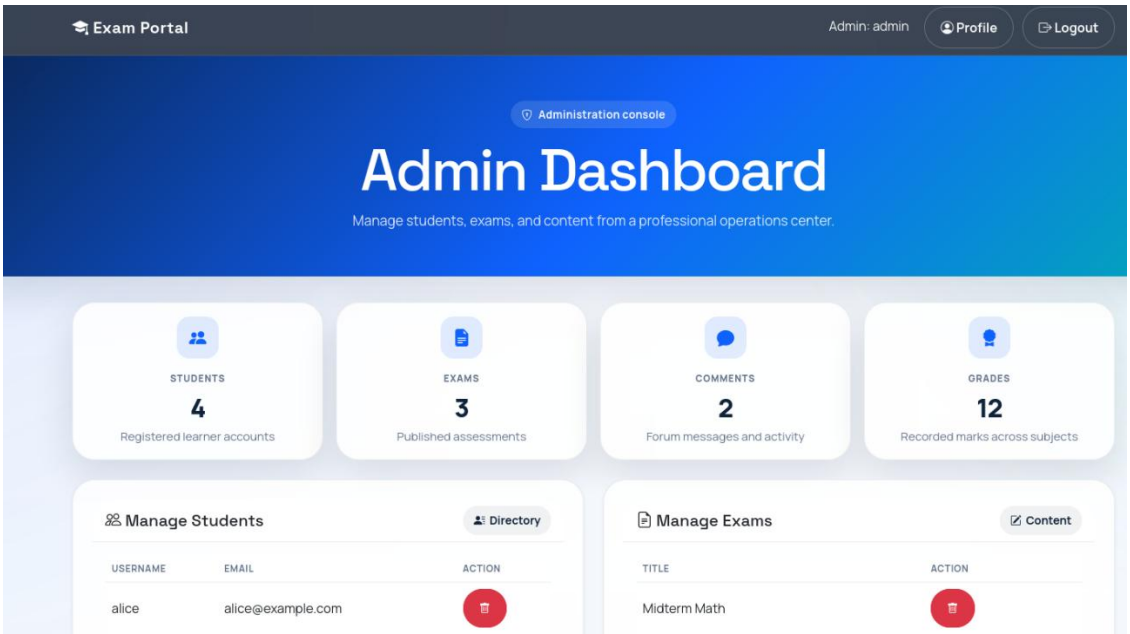
```
$query = "SELECT * FROM users WHERE username='$username' AND password='$password'";
```

When the username is admin' -- , the injected single quote closes the username string, and the -- sequence turns the rest of the query, the entire password check, into a comment. The query effectively becomes

"select the user whose username is admin" with the password condition removed entirely, which grants access without knowing the password.



Login form with the SQL injection payload admin' -- entered in the username field.



Logged in as an admin, the dashboard reached after the authentication bypass.

5.2 SQL Injection: Student Profile (GET) with sqlmap

Location: student.php, the public student profile page (id URL parameter). No login required.

Type: URL-based SQL injection, exploited manually and with sqlmap.

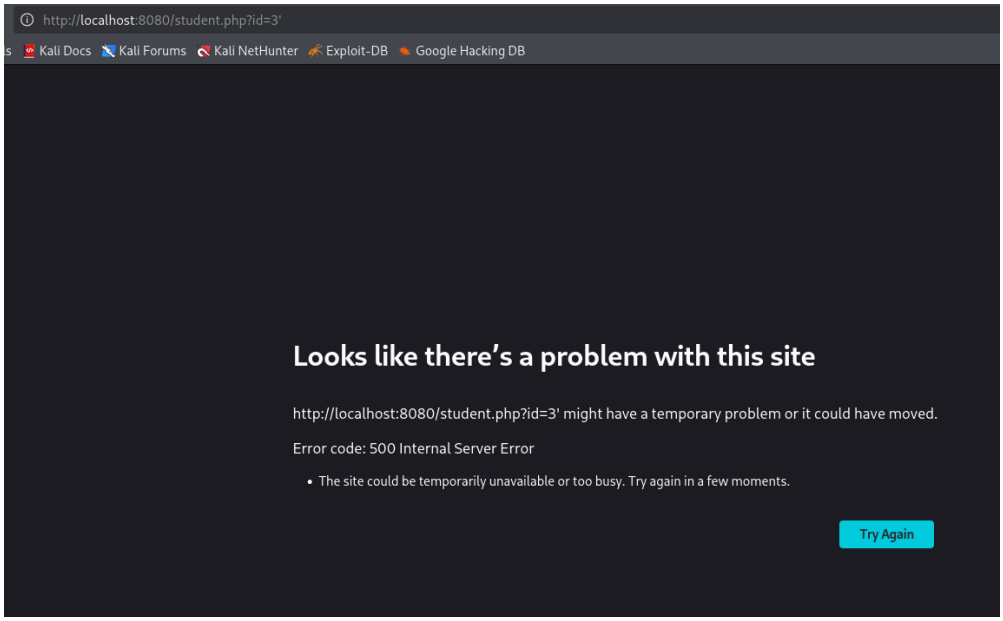
Manual probe: a single quote is added to the student ID to test whether the parameter is injectable:

```
http://localhost/student.php?id=1'
```

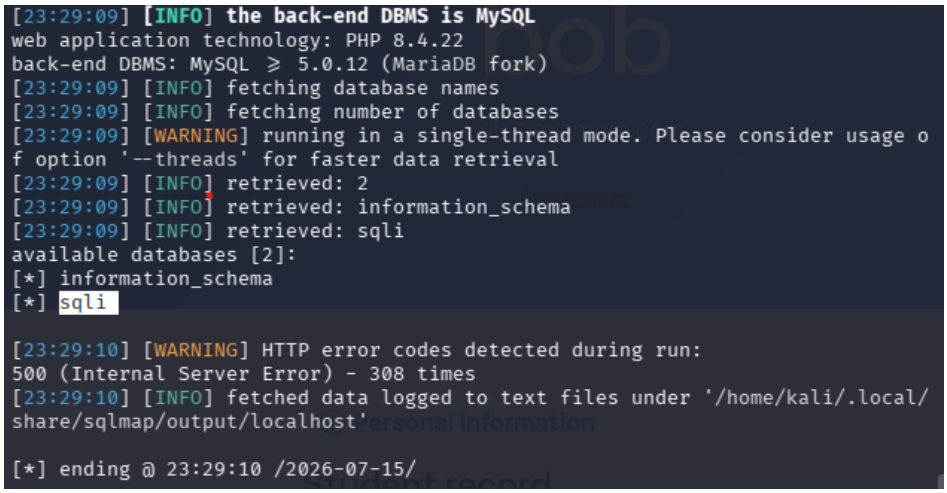
A database error is returned, confirming the input reaches the query unfiltered.

Automated exploitation with sqlmap: sqlmap automates detection and data extraction. A plain, valid ID is supplied and sqlmap injects its own payloads:

```
sqlmap -u "http://localhost/student.php?id=1" --dbs
sqlmap -u "http://localhost/student.php?id=1" -D sqli --tables
sqlmap -u "http://localhost/student.php?id=1" -D sqli -T users --dump
```

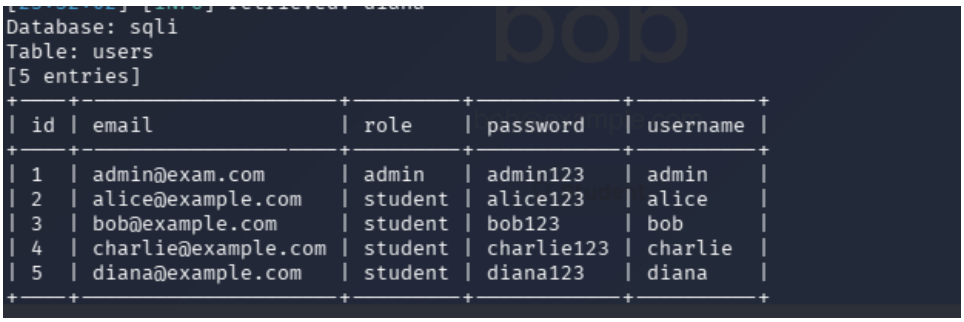


A single quote (id=3') triggers a raw MySQL SQL error, confirming the parameter is injectable.



sqlmap enumerating the available databases, information_schema and sqli.

sqlmap listing the tables inside sqli: users, exams, comments, and grades.



sqlmap dumping the users table, every username and plaintext password is exposed.

Why it works: the student ID is concatenated straight into the SQL query without sanitisation or parameterisation:

```
$query = "SELECT * FROM students WHERE id=$id";
```

Because the parameter is passed in the URL, an attacker can craft requests freely. sqlmap uses the injection point together with the database's built-in information_schema to enumerate the structure, then dumps the users table, exposing every username and password.

5.3 Stored Cross-Site Scripting (Stored XSS)

Location: payload stored via discussion.php (comment field); it executes in discussion.php whenever any user opens the forum.

Type: persistent (stored) XSS that executes for every user viewing the forum.

Payload (entered in the comment field):

```
<script>alert('Stored XSS')</script>
```

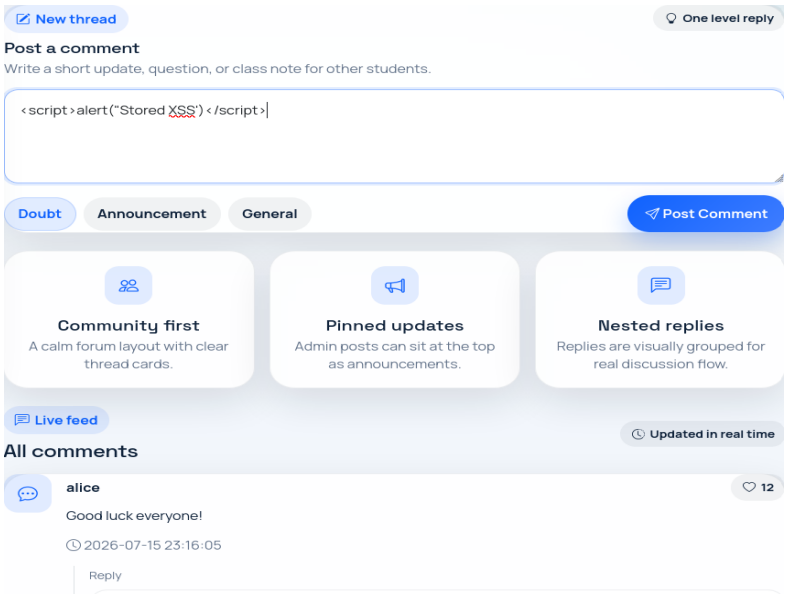
Steps:

- Log in as a student (e.g., alice / alice123) and open the Discussion Forum (discussion.php).
- Paste the payload above into the "Post a comment" textarea, then submit. The payload is now stored in the database.
- Refresh the page or log in as another user. The stored script executes inside the viewer's session.

Why it works: the comment is saved to the database. mysqli_real_escape_string() is used to prevent SQL injection on the insert, but the output rendering path does not escape HTML:

```
echo $row['comment'];
```

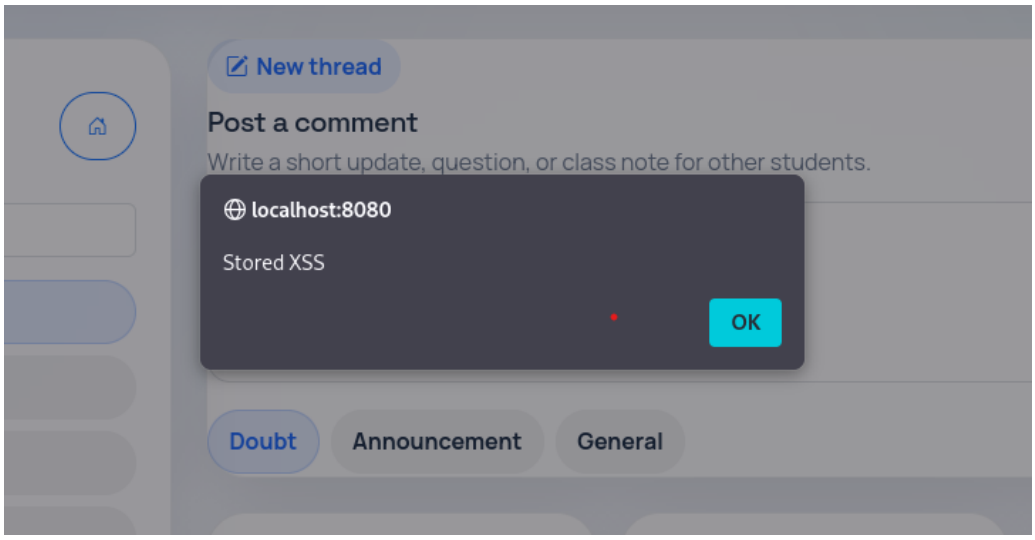
Because the value is never passed through a function such as htmlspecialchars(), the browser reads the stored <script> tag as executable code rather than plain text. Since the payload is stored, it fires every time someone opens the forum.



A student posts a comment with the <script>alert('Stored XSS')</script> payload in the textarea

id	user_id	comment	posted_at
1	2	Good luck everyone!	2026-07-15 23:16:05
2	3	The midterm was fair.	2026-07-15 23:16:05
3	2	<script>alert("Stored XSS")</script>	2026-07-15 23:36:10

The payload is stored verbatim in the comment column of the comments table in the database.



When the forum page loads, the stored script executes, triggering the JavaScript alert box.

5.4 Reflected Cross-Site Scripting (Reflected XSS)

Location: index.php, the homepage search box (q parameter).

Type: reflected (non-persistent) XSS; the payload is echoed back from the request and executes immediately.

Payload:

```
<script>alert(document.cookie)</script>
```

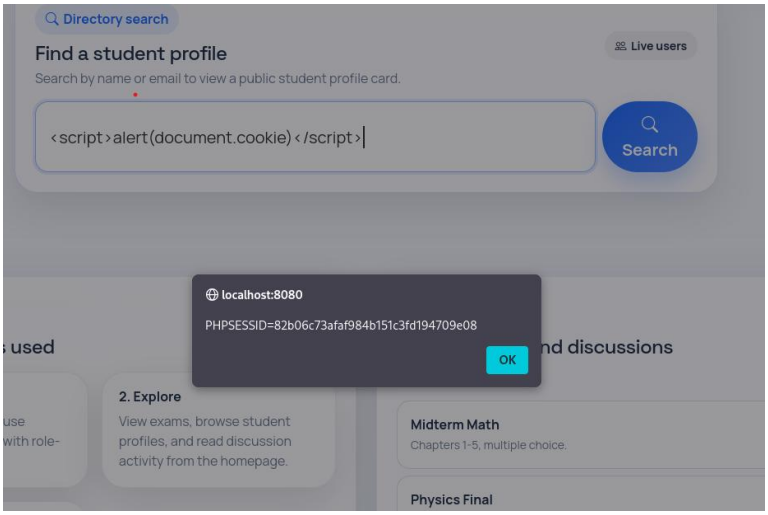
Steps:

- Navigate to the homepage search bar.
- Enter the payload into the search box and submit, or craft the URL directly.
- The search term is echoed back into the results heading without encoding, and the script executes.

Why it works: the user-supplied search value is written directly into the HTML response without output encoding:

```
echo "Results for: " . $_GET['q'];
```

Because the value is never encoded with htmlspecialchars(), a <script> payload in the request runs as live code. Unlike stored XSS, nothing is saved to the database here; the payload travels through a crafted link. An attacker can send that link to a victim, and the script runs in the victim's session as soon as they click it.



Reflected XSS on the homepage search, the injected script runs and displays the user's session cookie or triggers an alert.

5.5 Cross-Site Request Forgery (CSRF)

Location: profile.php (update email) and delete_exam.php (delete an exam). Neither endpoint uses an anti-CSRF token.

Type: CSRF, forcing a logged-in user's browser to perform a state-changing action without their intent.

Two state-changing actions are exploited: updating a profile email (POST) and deleting an exam (GET).

(a) Forcing a profile email update (POST)

A malicious page silently submits a hidden form to the profile update endpoint:

```
<form action="http://localhost/profile.php" method="POST" id="csrf">
  <input type="hidden" name="email" value="attacker@evil.com">
</form>
<script>document.getElementById('csrf').submit();</script>
```

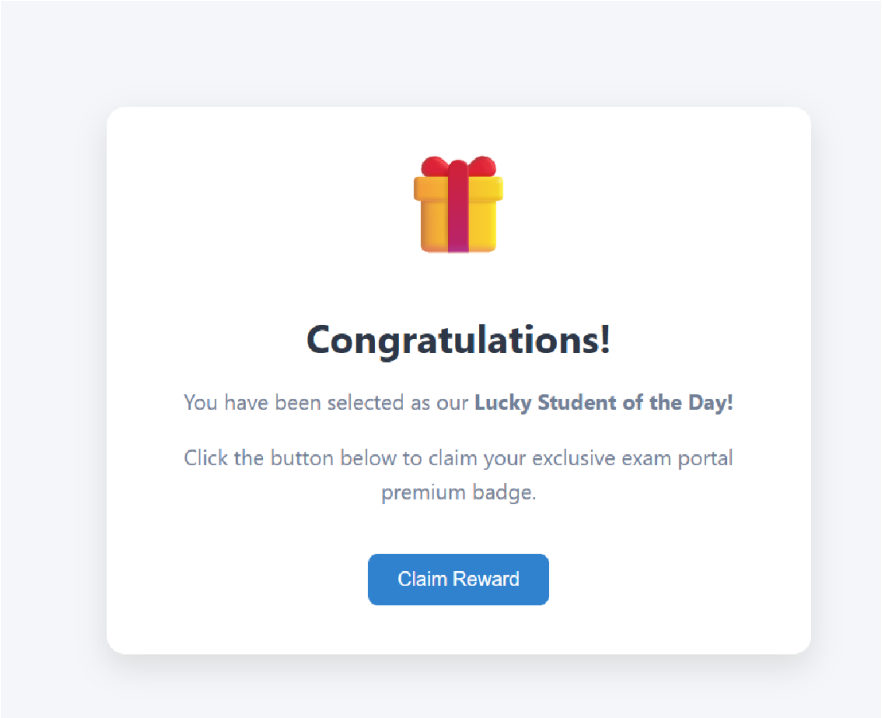
- Ensure a student is logged in to the portal in the browser.
- The student opens the attacker's malicious page.
- The hidden form submits automatically to profile.php using the student's session.
- Returning to the portal, the student's email has been changed to the attacker's email without their consent.

(b) Forcing an exam deletion (GET)

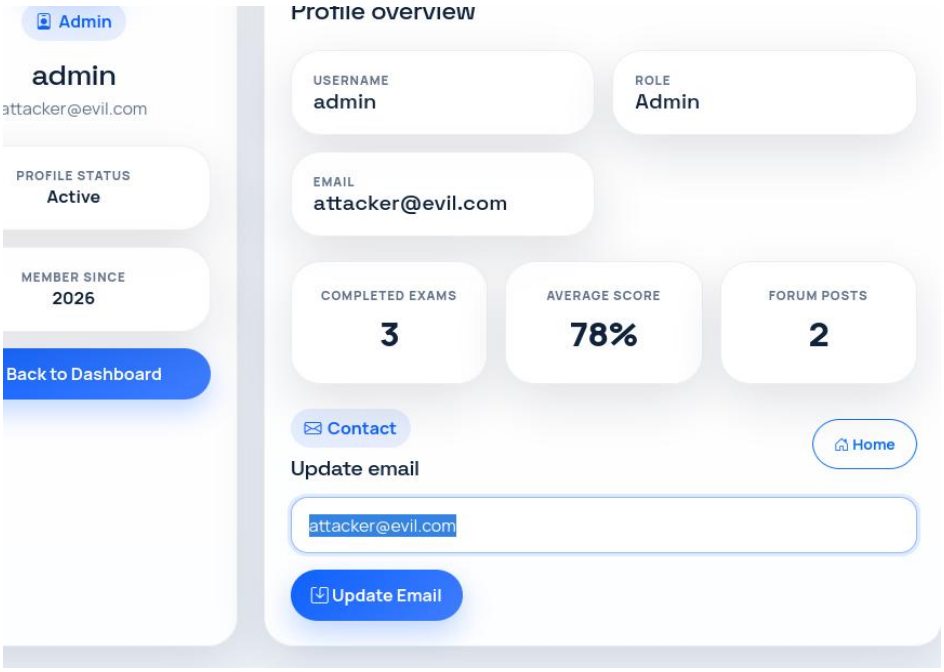
A more damaging variant forces the admin's browser to delete an exam permanently, using a plain image tag:

```
<img src="http://localhost/delete_exam.php?id=1" width="0" height="0">
```

- With an admin logged in, note an existing exam on the dashboard.
- The admin opens the malicious page. The browser automatically requests the delete URL, carrying the admin's session cookie.
- Returning to the dashboard, the exam has been permanently removed.



The attacker's page, a harmless-looking decoy served from a different origin.



After visiting the malicious page, the profile page confirms the email was updated

Why it works: the state-changing endpoints never verify that the request came from a legitimate form on the site; there is no CSRF token anywhere. The server only checks that a valid session cookie is present,

and the browser attaches that cookie automatically to any request sent to the domain. So a request forged from an external page is accepted as genuine.

6. Conclusion

This project set out to find and exploit four vulnerabilities inside the Online Examination Portal: SQL Injection, Stored XSS, Reflected XSS, and CSRF. All four were exploited successfully, and they come down to the same mistake: the application trusts input it has no business trusting.

The SQL injection flaws come from building queries by concatenating user input instead of using parameterised statements. Both XSS flaws come from writing user input into a page without encoding it. CSRF comes from accepting a state-changing request without checking where it came from, since there is no anti-CSRF token anywhere in the app.

What stood out while working through this is how ordinary each entry point was: a login box, a search field, a comment form, an admin delete link. None of them look dangerous on their own. The fix in every case is unglamorous and well known: parameterised queries, output encoding with htmlspecialchars, and anti-CSRF tokens. Skipping any one of them is what turned these routine features into open doors.